Problem Set 1: Getting Started

6.4210/2 Robotic Manipulation — Fall 2026

Due: September 17, 2026

Out: September 10 **Due:** September 17

Relevant reading in the course notes:

- Ch. 1 — Introduction
- Ch. 2 — Let's get you a robot

The homework assignments in this class have the primary objective of *teaching*, not assessment. The real assessments are the quizzes and the final project report and video. Use these assignments to help focus your study and to help you interrogate your own understanding.

In general, each "bookwork" question comes in two pieces: an immediately graded part, marked **Gradescope**, whose answer is a choice or a literal number or word that you enter in Gradescope; and a more open-ended part, marked **Going further**, that you should engage with to interrogate your own understanding. We will hand out solutions to these parts, and the exams will expect this depth of understanding.

The "computational questions" involve getting simulated robots to perform various tasks. You are welcome to use any tools you wish to do these questions. However, it's important that you remain engaged with and critical of the process. We will ask you to (1) provide some "checkpoint" answers to concrete questions about the results of your code in gradescope and (2) to make an upload a movie of your robot working.

After the computational questions comes a set of reflection questions indicating our expectations for what you should understand about your code, however it was produced.

1 Bookwork

Problem 1 State

The *state* of a system is the minimal information about its past that is sufficient to determine its future states and outputs, given future inputs.

Gradescope: Think about what state is needed for each of the following dynamical systems. Enter as an integer how many dimensions each state has.

1.1 A pendulum in the plane. Dimension: **2** [1]

Solution: *Angle and angular velocity, so 2.*

1.2 A free body in 3D. Dimension: **12** [1]

Solution: *At this point we can be informal: 3D position, 3D rotation, and then 6D velocity, so 12. We'll find that strategies for representing rotations in 3D are tricky and often use more than 3 numbers (although in some fundamental sense there are only 3 degrees of freedom).*

1.3 A mobile robot in the plane that discharges its battery as it moves, and can re-fill it at certain locations. Take it to be slow enough that its velocity can be ignored. Dimension: **4** [1]

Solution: *Position (x, y, θ) and battery level, so 4. For mobile robots that are relatively slow we can sometimes ignore velocity, but in general it may still matter.*

Going further: What if you bolt the pendulum's pivot onto a cart that you can drive back and forth along a horizontal rail. By how much does the state grow, and why? What if instead you leave the pivot fixed but replace the rigid rod with a spring that can stretch along its length?

Solution: *Each change adds one way for the system to move, and so two numbers: the cart's position and speed along the rail, or the rod's length and its rate of change. Both take the state from 2 to 4.*

Problem 2 Configuration space

Gradescope: For each of these robots, give the dimension of its configuration space and the dimension of its actuation space.

2.1 A two-wheeled differential-drive mobile robot. [1]
Configuration: **3** Actuation: **2**

Solution: *A 3D configuration space (x, y, θ) but a 2D actuation space: it is underactuated, which is why it cannot slide sideways into a parking space.*

2.2 A Kuka IIWA manipulator. [1]
Configuration: **7** Actuation: **7**

Solution: *7 dof configuration space and actuation space. It is "fully actuated".*

2.3 A flying drone with four fixed propellers. [1]
Configuration: **6** Actuation: **4**

Solution: *6 dof configuration space and 4d actuation space — underactuated again, which is why it has to tilt in order to translate.*

Going further: Here are two cases with more complicated answers.

2.4 A snake.

Solution: *This is sometimes called a "continuum robot" — there is no clear finite integer number of degrees of freedom, and the actuators are muscles, which are just as hard to count. You pick a finite-dimensional approximation to suit the task.*

2.5 An articulated ("bendy") city bus: two rigid sections joined by a vertical hinge, with the rear section driving and the front section steering. What is the dimension of its configuration space now, and what is actuated?

Solution: *4: the planar pose of the front section (x, y, θ) plus the hinge angle. Only 2 are actuated (drive and steer), and the hinge angle is not commanded at all — it is whatever the history of the motion has induced.*

Problem 3 Manipulation [1]

Gradescope: What makes robot manipulation fundamentally different from robot driving?

A. Perception is not a key issue in robot manipulation.

B. Manipulators have more degrees of freedom than mobile robots.

C. Manipulation does not involve interaction with people.

D. Robot manipulation is about changing the environment through selective contact, while driving requires always avoiding contact with the environment (beyond the wheels and road).

Solution: *For a robot manipulating the world around it, the size of the configuration space includes the configurations of all the objects as well as the robot, so it's generally more complex to reason about. In addition, the physical interaction between a robot and objects it is contacting can be difficult to model and predict.*

Going further: What about a snowplow or a robot vacuum that can nudge chairs out of its way? Are these systems more like automated cars or like manipulation robots?

Solution: *More like manipulators, because they have to change the world state and reason about the effects of their interactions with other objects (or stuff) in the world.*

Problem 4 Manipulation systems [1]

Gradescope: Which of these describe prominent approaches to the design of robot manipulation systems? (Select all that apply.)

- √ **End-to-end imitation learning (pixels to motor commands)**
- √ **End-to-end reinforcement learning (pixels to motor commands)**
- √ **Sense-Plan-Act architectures**
- √ **Scripted sequences of motion commands**

Solution: *All four. The field hasn't settled on an answer and the last one is old-school but still definitely in use.*

Going further: Pick two of these four approaches and think about the resources they need: a simulator, demonstrations, object models, robot models, engineering time. How do they trade off against each other?

Solution: *Roughly: imitation learning needs demo data (but not models or many much engineering), RL needs a simulator (hence models, but not labeled data), sense-plan-act needs robot and world models (or good perception), scripts require a lot of engineering work.*

Problem 5 Joint types

Gradescope: For each of the mechanisms below, indicate what kind of joint(s) it has.

5.1 Kuka IIWA main joints **A. revolute** B. prismatic C. fixed D. free [1]

5.2 A cupboard drawer A. revolute **B. prismatic** C. fixed D. free [1]

5.3 A parallel-jaw gripper A. revolute **B. prismatic** C. fixed D. free [1]

5.4 A block resting on the table A. revolute B. prismatic C. fixed **D. free** [1]

Solution: *"Free" (6 dof), not "fixed". Nothing in the model constrains the block; the table holds it up with contact forces, which the physics engine computes, rather than with a joint.*

Going further: What kind of joint is the lid of a screw-top jar, or a bolt going into a threaded hole? What about a shoulder? How many degrees of freedom does each have, and can you build either one out of the four joint types above?

Solution: *A screw has 1 dof, but it couples rotation and translation in a fixed ratio, so it is neither revolute nor prismatic; Drake has a separate* `ScrewJoint` *for it. A shoulder is a ball joint with 3 rotational dof, Drake has a* `BallRpyJoint` *or you could approximate it with three intersecting revolute joints.*

Problem 6 Drake [1]

Gradescope: Drake is a large software library. What are some important components? (Select all that apply.)

- √ **Block diagram specification of systems, based on the "signals and systems" abstraction**
- √ **Extensive optimization toolkit**
- √ **A sophisticated "physics engine" for multi-body dynamics**

◯ GPU parallelized simulations

√ **Deterministic replay of simulations**

Going further: Why do we care about deterministic replay of simulations?

Solution: *It really lets you debug something until it's exactly and reliably right.*

2 Computational questions

Initial Setup and Installation

Follow the setup instructions in the README of the handouts repository:

https://github.com/tlpmit/6.4210-2-fall26-handouts

It covers installing the prerequisites (git, Python via uv, Drake, Graphviz), cloning the repository, keeping your ongoing work in a *private* GitHub repository, pulling updates when psets are released or revised, and working on Google Colab instead of a local machine.

Problem 7 Discrete-time systems in Drake

Drake offers a very sophisticated set of tools for defining system blocks and wiring them up. In this lab we are going to build up a simple interface for discrete-time systems. It assumes the input, state, and output of the system can each be specified as a vector of real values, and that the operation of the system is captured by two functions:

- the **next-state function** computes $s_{t+1} = f(s_t, i_t)$, where s_t is the state at time t and i_t is the input at time t;
- the **output function** computes $o_t = g(s_t, i_t)$.

As you read the provided code and write your own functions, read the Drake C++ Documentation (we are writing in python, and there is Drake Python Documentation, but most of the C++ functions are also bound in python and the python documentation is mostly auto-generated from the C++ documentation, which is more carefully curated.) It can be helpful to add debugging print statements to understand what each line of code does. Also, use `plot_system_graphviz`, to draw a system's block diagram: each subsystem as a box listing its input and output ports, with an arrow for every connection.

7.1 **A free body in 1D** Read the code for `DTVectorSystem` in `dtsystems.py`. Use it to implement a system representing a free body in 1D whose input is a force and whose output is the position of the body. Concretely, write the body of the function `Body1D` in `dtsystems.py`. [5]

Use the forward Euler integration method: $s_{t+1} = s_t + \Delta t\, \dot{s}_t$.

Use the `Constant` and `Counter` constructors that are defined in the code as examples of defining systems, and `main0` as an example of how to instantiate and simulate and add logging to a system.

Gradescope: Use `next_state_fun` on your `Body1D` to compute the state after a single update of timestep 0.1 s, for a body of mass 2 kg, from an initial position of 0.0 and a velocity of 0.5, with an input force of 0.3 N.

Position: **0.05** Velocity: **0.515**

Solution: *Write the state as (q, v), position and velocity. Forward Euler on $\dot{q} = v$ and $\dot{v} = f/m$ gives*

$$q_{t+1} = q_t + \Delta t\, v_t, \qquad v_{t+1} = v_t + \Delta t\, \frac{f_t}{m}.$$

With $q_0 = 0.0$, $v_0 = 0.5$, $\Delta t = 0.1$, $m = 2$ and $f_0 = 0.3$: $q_1 = 0.0 + (0.1)(0.5) = 0.05$ and $v_1 = 0.5 + (0.1)(0.3)/2 = 0.515$. Note that q_1 uses the old *velocity v_0 — that is what makes it forward Euler.*

7.2 **Series composition** Read the code for `series_composition`, which takes two systems and composes them by wiring the output of the first into the input of the second. Use the `Constant` constructor to make a constant force source, couple it with the body system you defined above, and use `simulate` to see where the body is after 5 steps of applying 0.1 N. Note that `simulate` takes a duration in *seconds*, not steps, so with `dt` = 0.1 you want `T` = 0.5. [5]

Use the same body as in part 7.1 — mass 2 kg and `dt` = 0.1 s — and start it at rest at the origin, so its initial state is $(0.0, 0.0)$. `simulate` returns the simulator; pass the body and the simulator to `get_state` to read out the body's final state. Use the `plot_log` helper function to plot the output of the `Body1D` system.

Gradescope: Position: **0.005** Velocity: **0.025**

Solution: *Five forward Euler updates of $q_{t+1} = q_t + \Delta t\, v_t$ and $v_{t+1} = v_t + \Delta t\, f_t/m$, with $\Delta t = 0.1$, $m = 2$, $f_t = 0.1$ and $(q_0, v_0) = (0, 0)$. The acceleration is $f/m = 0.05$, so $v_t = 0.005\,t$ and $v_5 = 0.025$, while the position accumulates the old velocities: $q_5 = 0.1\,(0 + 0.005 + 0.010 + 0.015 + 0.020) = 0.005$. Using an exact integration scheme, the position would be $\frac{1}{2}(f/m)T^2 = 0.00625$; forward Euler results in a smaller value because each step uses the velocity from the start of the step.*

7.3 **A position controller** Given a desired position of the body, a position controller can be used to modulate the applied force to get the body to the right position, even in the presence of some amount of disturbance. [5]

Use `DTVectorSystem` to implement a position controller with the current position as input and the force as output, where the force is proportional to the difference between the current and desired position. Write this generally, so we can use it later.

Gradescope: Use `output_fun` on your `PController` to compute the force it commands. Take a controller of dimension 1 with a set point of 1.0 and a gain of 10, and an input (the body's current position) of 0.2.

Force: **8.0**

Solution: *A proportional controller is memoryless, so its output depends only on the input: $f = k\,(q^* - q) = 10\,(1.0 - 0.2) = 8.0$. The state argument is ignored — `PController` declares no state and passes `None` as its next-state function.*

7.4 **Feedback composition** Define `feedback_composition` which, like `series_composition`, takes two systems, but this time connects the output of each one into the input of the other. This resulting system will have no inputs or outputs. [5]

Use it to combine your controller with your body. Give it a target position of 1.0 and simulate and log the behavior of the combined system. Try some different gains.

Gradescope: Combine your `PController` with your `Body1D` using `feedback_composition`: mass 2 kg, `dt` = 0.1 s, gain 10, and a target position of 1.0. Start the body at position 0.2 with velocity 0.5, simulate for `T` = 10 s, and plot its position. Which best describes the behavior?

A. It settles at the target position of 1.0.

B. It oscillates about 1.0 forever, with constant amplitude.

C. It oscillates about 1.0 with growing amplitude.

D. It moves away from 1.0 without ever oscillating.

Solution: *Unstable: the oscillation grows due to compounding errors from the forward Euler integration. Adding a damping term (like the derivative term in the PD controller, or a joint damping term) can fix this.*

Problem 8 A 2D Cartesian robot

Now, we'll play with a simple "robot" with 2 prismatic joints. This robot can be moved in x and y, and is rendered in Drake as a vertical cylinder on a table.

Read the code in `cartesian_2d_robot.py` to see how we define a table (no legs) as an SDF object and attach it permanently (weld it) to the world. Then we define the "finger" as a more complicated body, with inertia and damping, and two `PrismaticJoint`s.

8.1 **Drive the finger with a constant input** The procedure `create_2d_robot_diagram` uses Drake tools to construct a scene with the table and the finger and put them into a physical system called a `MultibodyPlant`. [1]

Gradescope:

Run `test_const_input` with input $[4.0, 1.0]$ and interpret the plot of the state of the finger. Why do the x and y velocities plateau?

A. The input to the system is a constant velocity command.

B. There are velocity limits defined on the prismatic joints in the SDFs.

C. Friction between the bottom of the finger and the table surface grows until it balances the input force.

D. There is damping defined on the joint proportional to the velocity of the finger which balances the input force.

E. Gravity pulling down on the finger balances the input force.

Solution: *Each prismatic joint declares a `damping` of 10 N s/m, which `PrismaticJoint` models as a force $f = -b\,v$ opposing motion. So the finger obeys $m\ddot{x} + b\dot{x} = F$, not $m\ddot{x} = F$: the drag grows with speed until it cancels the applied force, leaving a terminal velocity of F/b: $4/10 = 0.4\,m/s$ in x and $1/10 = 0.1\,m/s$ in y. None of the others apply: The SDF does not set a velocity limit and a `MultibodyPlant` only enforces position limits during simulation but not velocity limits. No collision geometry is defined in the SDF, only visual geometry so there would be no friction and even if collision geometry were defined, there is a 1 mm gap between the geometry of the finger and the table so they would never touch. Gravity acts along $-z$, perpendicular to both joint axes.*

8.2 **Drive the finger to a set position** Modify the code to use your position controller to drive the finger to a set position (have it take the desired position as input). One problem: the plant's state output port carries $[x, y, \dot{x}, \dot{y}]$, but the controller needs just the two positions. Write the body of `Observer` in `dtsystems.py` — a stateless system whose output is the elements of its input selected by `indices` — and compose it between the plant and the controller. Like the controller, it is written generally: we will use it again on the iiwa. [5]

Gradescope: With the SDF settings already defined in the script and a position controller gain of 100, initial finger state defined by `finger_s0(FINGER_START)`, and a target position of $[0.9, 0.6]$, how long does it take for the finger to come to rest at the target position? (Use `plot_log`, and/or look at the meshcat recording)

Solution: *About 1 s. (gradescope accepts 1.3 ± 0.5)*

8.3 **A trajectory follower** Next, we would like to drive the finger through a series of waypoints, by defining a sequence of configurations and using the position controller to get within some epsilon of the first one, then switching to the next one, and so on. [5]

Write a `DTVectorSystem` that takes as input the current position of the robot and outputs a setpoint for the controller. It will have to use its state to keep track of which waypoint it is working on.

Write the body of `PController2` which is a new version of `PController` that takes the desired set-point as well as the current value as input.

Use these components to build a system that drives the robot in a `SQUARE`. Try different values of gains and epsilon.

Gradescope: The script defines `JOINT_DAMPING` = 10, and it is baked into the SDF text when the file is loaded — so for this part, edit `cartesian_2d_robot.py` to set `JOINT_DAMPING` to 20 and rerun. Use a controller gain of 100. Drive the robot around `SQUARE` for `T` = 5 s. How far does the finger travel? (Use `plot_path`, which reports the length of the path it plots.)

$\epsilon = 0.1$: **2.2520** m $\epsilon = 0.01$: **2.3768** m

Solution: *The square's four 0.6 m sides total 2.4 m, and both runs come in under that: the follower switches to the next waypoint as soon as it is within ϵ of the current one, so it rounds every corner instead of reaching it. The larger ϵ cuts more corner, so it gives the shorter path.*

Problem 9 Simulating the IIWA

Now we can apply these same ideas to a real robot. Read the code in `iiwa_1.py`. We can apply the controllers and basic strategy for doing things with the robot that we developed on the simple robot now to the real Iiwa!

9.1 **Zero torque** Start the robot at configuration $[0, 1.0, 0.3, 0.7, 0, 0, 0]$, defined for you as `Q_START`. [1]

Gradescope: Using `test_const_torque`, what happens if you apply a constant input torque of 0 to each joint?

A. The arm stays at the start configuration: with no input, nothing moves it.

B. The arm drifts slowly away from the start configuration as numerical integration errors accumulate.

C. The arm swings down back and forth under gravity like a multi-link pendulum.

D. The arm collapses straight down and hangs motionless within a second.

Solution: *Nothing holds the arm up, so it falls out of the start configuration and swings like a multi-link pendulum. The only resistance is the 0.5 N·m·s/rad of viscous damping the iiwa URDF declares at each joint, so it is still swinging when `test_const_torque`'s 5 s run ends.*

9.2 **Constant torque** Start the robot at configuration `Q_START`. [1]

Gradescope: What happens if you apply a constant input torque of 20 N·m to each joint?

A. The arm holds its start configuration: the applied torque balances gravity.

B. Every joint spins around at a constant rate, forever.

C. The arm behaves exactly as it does with zero torque: 20 N·m is too small to matter against gravity.

D. The joints wind up and jam against their position limits.

Going further: What would happen on a real robot?

Solution: *On the real robot, the iiwa's low level controller would emergency stop (E-stop) the arm, applying brakes and stopping the arm from moving before it reached its mechanical position limits.*

9.3 **A proportional controller** Now we will add a proportional controller to the iiwa. [5]

One problem is that the Drake state for this simulation is 14-dimensional (positions and velocities). Use the `Observer` you wrote for 8.2 to map the 14D state into a 7D position observation which is needed as input to your `PController`.

Gradescope: Use the controller, with a gain of 100, to try to maintain the original configuration for 10 seconds. Which best describes the behavior?

A. The arm stably holds the start configuration.

B. The arm oscillates about the start configuration with growing amplitude.

C. The arm oscillates with decreasing amplitude but around a gravity sagged pose.

D. The arm falls exactly as it does with zero input torque.

Solution: *Gravity is a constant disturbance torque on each joint. Unlike problem 7's point mass, the loop does not go unstable, because the URDF's joint damping dissipates energy. A larger gain (e.g. 10000) allows the arm to stay much closer to the start configuration, but it continues to oscillate.*

9.4 **A PD controller** Later in the class, we will study more sophisticated controllers in detail. But, for now, try out the PD controller that is defined for you in `dtsystems.py`. How does it affect the system's behavior? [5]

It estimates velocity by differencing successive position measurements, so its `dt` argument is the rate at which the controller itself runs; make sure you pass one that suits the simulation rather than relying on the default.

Gradescope: With a controller gain of 100 and a damping gain of 30, drive the arm from `Q_START` to `Q_START` + `Q_STEP`, where `Q_STEP` $= [0.5, -0.3, 0, 0.4, 0, 0, 0]$ (defined for you in `iiwa_1.py`) — a step on joints 1, 2 and 4. How long does it take for the arm to come to rest? (Use `plot_log`, and/or watch the meshcat recording.)

Solution: *About 3 s: the joint position traces are flat from then on. (Gradescope accepts 3 ± 2.) Note the position the arm comes to rest at. The derivative term removes part 9.3's oscillation, but not the sagging due to gravity.*

9.5 **IIWA Trajectory** Adapt your trajectory follower for the IIWA. Use it to track `SQUARE_IIWA` (initialize the position to be the first waypoint), a sequence of joint configurations given in `iiwa_1.py` whose end-effector positions are the corners of a 0.4 m square in a vertical plane in front of the robot, with the end-effector frame held axis-aligned (we found them with inverse kinematics, a tool we will study later in the term), so that the end effector draws the square. [5]

Use proportional gain of 10000 and derivative gain of 3000, and epsilon of 0.01. Play the simulation back in your browser and upload a screen recording of it to Gradescope (an mp4 file, much less than 500MB). Gradescope should indicate "Submitted"; the points for this problem will be assigned when the staff grades it.

Solution: *Gains of 10000 (P) and 3000 (D) draw a recognizable square while the edges bow by a few centimeters, because straight lines in joint space are not straight lines in task space (something else we will fix later in the term). Part (d)'s gains of 100 and 30 do* not *work here: the follower only advances once the arm is within ϵ of the current waypoint, sag caused by gravity prevents it from reaching even the first corner.*

2.1 Additional problems for 6.4212

For each question below submit a PDF which has a few sentences and at least 1 plot. Each answer should not be more than 1 page long (including plots).

Problem 10 Step size 6.4212 only [10]

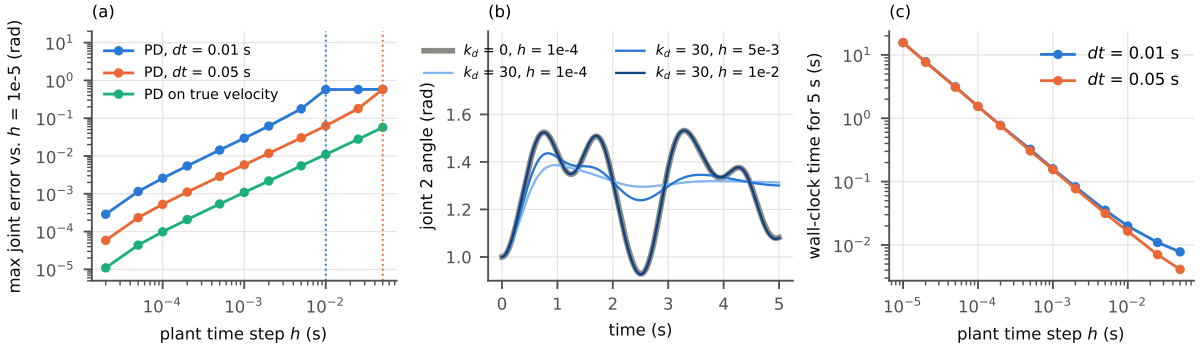
There are two step sizes which impact the behavior we observe in the simulations for this pset: the plant's integration time step, and the controller's update period. How does changing the value of one or both of these step sizes impact the simulation?

Some ideas (feel free to explore this in other directions, and you don't need to answer all these questions):

1. How do these two step sizes impact/relate to each other?
2. Do you observe particular failure modes at certain values and what causes them?
3. Vary the plant's time step and use the smallest time step as the ground truth and compute error as the time step increases.
4. How do these step sizes impact wall-clock-time for computing the simulation?

Solution:

Setup. *We repeated the step response of part 9.4 (gains $k_p = 100$ and $k_d = 30$, from `Q_START` to `Q_START + Q_STEP`, 5 s), varying the plant time step h and the controller period dt. As in `test_pd_controller`, the controller's stored measurement is initialized to the initial position. If it were initialized to zero, the first velocity estimate would be very large, and at large h the resulting torque would be applied for an entire plant step.*



(a) Each controller's error versus plant step h: the maximum joint-space distance over the 5 s simulation between its trajectory and its own trajectory at $h = 10^{-5}$ s, the smallest step, which we take as ground truth. Dotted lines mark $h = dt$. (b) Joint 2 at $dt = 0.01$ s. (c) Wall-clock time to simulate 5 s, excluding building the diagram.

How the two step sizes interact. *The controller has direct feedthrough, so $u = k_p(q^* - q) - k_d(q - s)/dt$ is recomputed at every plant step, and dt only sets how often the stored measurement s is updated. The derivative term starts each controller period at zero and grows as the arm moves away from s. Averaged over the period, the effective damping is $\frac{1}{2}k_d(1 - h/dt)$ for $h \le dt$, rather than k_d. Consistent with this, in (b) the oscillation decays more slowly as h increases toward dt (lighter to darker blue). For $h \ll dt$ the effective damping is about half the nominal value, regardless of dt: the simulation at the default step sizes ($h = 10^{-4}$ s, $dt = 0.01$ s) closely matches a controller that uses the true velocity with $k_d = 15$. When $h \ge dt$, the derivative term provides no damping: the controller updates s at least once per plant step, so whenever the plant evaluates the torque, s already equals the current position and $q - s = 0$.*

Error versus plant step. *When the controller uses the true velocity (green in (a)), all of its error is integration error caused by the `MultibodyPlant` advancing its state in discrete steps of h. It is first order in h (slope one in (a)), since each step is a first-order update. The finite-difference controllers (blue for $dt = 0.01$, orange for $dt = 0.05$) also have this integration error, but most of their error comes from the change in effective damping. The reference simulation at $h = 10^{-5}$ has damping of about $\frac{1}{2}k_d$, whereas a simulation at larger h has $\frac{1}{2}k_d(1 - h/dt)$. It therefore simulates a different, less damped controller, and both curves lie well above the green one. This component of the error depends on h/dt rather than on h: the orange curve is the blue one shifted right by a factor of five (the ratio of the two values of dt), so the two have the same error at the same h/dt. Integration error, which depends only on h, would not shift with dt. Once h reaches dt (the dotted lines), the effective damping is zero and both curves reach the same error, and the blue curve remains at that level for larger h.*

Failure modes. *At $h = dt = 0.01$ the simulations with $k_d = 0$, 30 and 100 are identical. The arm oscillates as it does under the P controller of part 9.3 (darkest blue and gray in (b)), dissipating energy only through the URDF's joint damping. The closed loop is not unstable, and the plant integrates it accurately; the controller simply provides no damping. With these low gains, even a long controller period such as $dt = 0.1$ has little effect on the trajectory. However, at the gains used for the square ($k_p = 10^4$, $k_d = 3000$) the derivative term acts as a spring of stiffness k_d/dt anchored at s, and the arm oscillates at about the controller period. The arm completes the square of part 9.5 at $dt = 0.01$ but not at $dt = 0.05$: it circles the first corner without coming within $\epsilon = 0.01$ of it.*

Wall-clock time. *The wall-clock time is proportional to $1/h$ (c), since each plant step takes about the same time. The controller period has little effect (the two curves in (c) overlap except at the largest h), since the controller's output is evaluated at every plant step regardless of dt, and a controller update only copies a vector.*

Problem 11 Damping 6.4212 only [10]

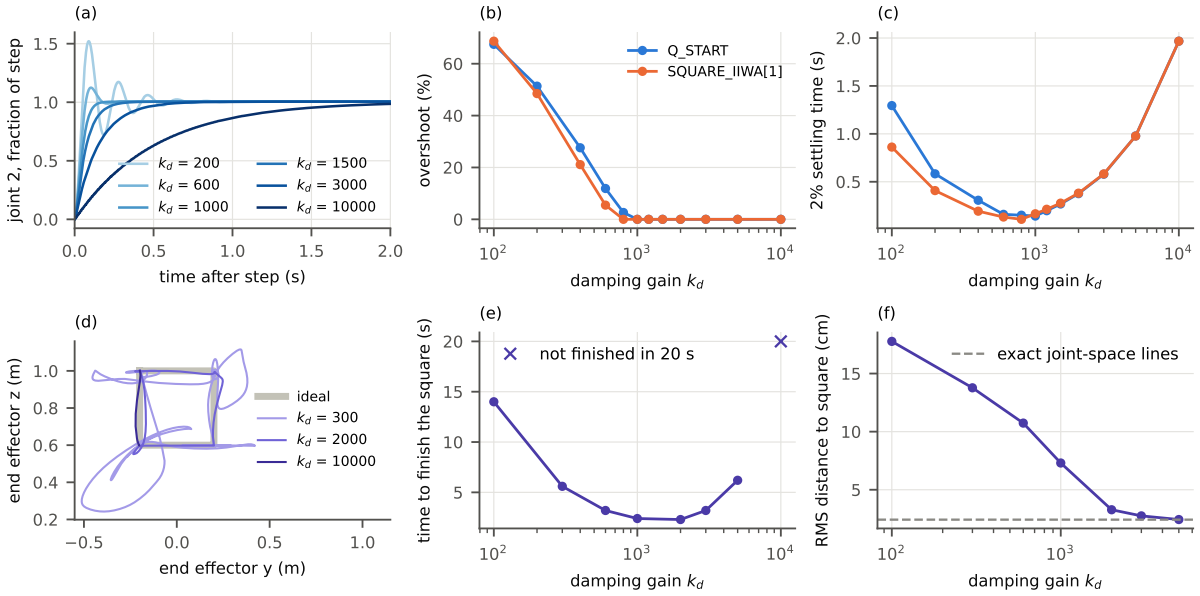
Experiment with the damping parameter in the PD controller. How does it affect the performance of the robot at following a trajectory?

Some ideas (feel free to explore this in other directions, and you don't need to answer all these questions):

1. Command a step change to a single joint and sweep the damping gain in the PD controller.
2. Identify responses that are under-damped, over-damped and critically damped.
3. How does overshoot and settling time vary with the gain?
4. Does the behavior depend on the arms configuration? Why, or why not?
5. Rerun the square-drawing task at several damping gains and plot the tracking error along the square and the time the arm takes to complete it. In what ways does more damping help or hurt?

Solution:

Setup. *All simulations use the proportional gain from the square task, $k_p = 10^4$, and the default step sizes, $h = 10^{-4}\,s$ and $dt = 0.01\,s$. For the step response, we let the arm settle under gravity at the starting pose and then step the setpoint of joint 2 (the shoulder) by 0.02 rad. We keep the step small so that the commanded torque stays under the shoulder's 320 N·m limit; a 0.3 rad step would command 3000 N·m, which Drake would clamp.*



(a) Step response at `Q_START`. (b, c) Overshoot and settling time at both poses. (d) End-effector path in the plane of the square. (e, f) Square task. (f) omits $k_d = 10^4$, for which the arm does not complete the square.

Under-, over- and critically damped. *With $k_d \leq 800$ the response is under-damped and overshoots the target (a, b). With $k_d = 0$ only the URDF's joint damping acts, and the arm is still oscillating when the simulation ends. The overshoot disappears at about $k_d = 1000$, close to critical damping, and above that the over-damped joint approaches the target slowly (the darker curves in (a), $k_d \geq 1500$).*

Overshoot and settling time. *Overshoot decreases as k_d increases and is zero above the critical gain (b). The 2% settling time is shortest near the critical gain and increases on either side of it (c). Under-damped responses oscillate before settling, and over-damped responses approach the target more slowly as k_d increases.*

Configuration dependence. *The critical gain depends on the pose through the shoulder's inertia M_{22}. At the square's second corner (`SQUARE_IIWA[1]`), the elbow is bent to 1.66 rad instead of 0.70 rad. This brings the forearm closer to the shoulder axis and lowers M_{22} from 8.0 to 5.7 kg·m^2. At this pose (orange in (b) and (c)) the overshoot disappears by $k_d = 800$, and the under-damped responses settle faster, because the same damping gain acts on a smaller inertia, so the oscillation decays faster. Over-damped responses settle at the same rate at both poses (the curves in (c) coincide): when damping dominates, the rate of approach is set by the gains and is nearly independent of the inertia. The gravity torque also*

depends on the pose, but at $k_p = 10^4$ it only shifts the steady-state position, by about 0.006 rad at the shoulder.

Square task. *Straight lines in joint space are curved in task space, so even perfect tracking of the joint-space path deviates from the square by 2.4 cm RMS (dashed line in (f)). More damping improves the tracking up to about $k_d = 2000$. With under-damped gains the arm overshoots every corner (d). It also takes longer to complete the square, because the follower checks only once every 0.1 s whether the arm is within ϵ of the waypoint, and advances only when a check succeeds. Above $k_d = 2000$, the tracking error keeps decreasing until the path matches the joint-space lines at $k_d = 5000$ (f), but the arm takes longer to reach each waypoint (e). At $k_d = 10^4$ the arm does not complete the square trajectory, with or without gravity. It never comes within ϵ of `SQUARE_IIWA[1]`, the first commanded corner, because the wrist joints (5 to 7), which have the smallest inertia, do not settle. With this much damping, the derivative term overcorrects these low-inertia joints within each controller period, so the wrist chatters at the controller rate with torque commands well beyond its limits.*

3 What you should understand about the code you just produced

- How to construct and add a new DTSystem module — not at the code API level, but at the content level: what are the states, update and output functions? What would the resulting diagram look like?
- You should be able to draw the system diagram for any system you build.
- Debugging skills: be able to add loggers to the diagram and plot their output, print the block diagram of systems, search and read the Drake documentation.
- Why `dt` enters the *math* of the physics simulation and of the PD controller, but not of the proportional controller (which is stateless and needs no `dt` at all) or of the waypoint follower (whose `dt` only sets how often it checks whether it has arrived).
- Why the controller gain matters.
- What role the declaration of the inertia of objects in an SDF file plays.
- What role the declaration of joint damping in an SDF/URDF file plays.
- How changing epsilon in the simple trajectory follower would change the path the robot follows. How does that interact with the controller gain?